M4GB on the Weil-descent curve ladder (2026-11-10)

M4GB on the Weil-descent curve ladder: solve time per target (2026-11-10)

Owner's ask: run the decomposition benchmarks on various curves and see whether solve time is lower with the new external backend. Follow-up to `research/m4gb_integration_20261110/` (backend `m4gb-f2`, patches, kernel A/B).

Status words. Everything here is a local development check on the session container (4 vCPU Intel Xeon 2.10 GHz, 16 GiB, shared with a Rust build at times), `[pcc:unverified]`. It prices one decomposition-oracle call, so by `AGENTS.md` §2 it is a stage diagnostic and never a speed row.

Instrument

`ic descent --solver …` (`src/bin/ic/descent.rs`, `price_engine_cell` in `ic_descent_degrees.rs`): for each cell `(family, n, n', m)` it builds the Weil descent of the Semaev polynomial over the same seeded targets, hands the Boolean system (`m·n'` unknowns, `n` equations) to every listed engine, interleaves the engines per target with the engine order rotating each repetition, and checks every decided answer against the reference engine (the strongest exhaustive search listed: `fes-f2` for the quadratic `m = 2` cells, `exhaustive` otherwise).

- Families: `K` (the Koblitz curve of degree `n`) and `R` (a random binary curve of the same degree, points counted exhaustively, which bounds `n`).
- Cells: `m = 2`, `n = 11, 13, …, 31` with `n' = ⌈n/2⌉` (12 to 32 unknowns); `m = 3`: `11:4:3`, `13:5:3`, `15:5:3`.
- Engines: `fes-f2` (reference), `f4-f2` (in-house F4, full reduced basis), `inherited-f4` (the Koblitz oracle's degree-bounded hybrid), `m4gb-f2` (M4GB upstream `01860a52` with patches 0001 + 0002, one thread, `M4GB_BINARY` a directory of binaries built for 16, 20, 24, 28, 30, 32, 35, 36, 40 and 64 variables; the narrowest that fits is used).
- Targets 6 per cell, repeats 2, budget 60 s per target and engine (a first attempt with 8 targets, 3 repeats and 300 s was killed by the session's two-hour cap before `ic descent` wrote its single output file; nothing of it survived, see the session log).
- One `ic descent` invocation per cell and family, each writing its own JSON under `results/`; for each family the `m = 2` ladder stops once `m4gb-f2` exhausted the budget on every target of a cell.

- Launcher: run_ladder.sh, run under `tools/benchrec.py run` (record `20261110T1513Z-m4gb-descent-ladder`).

Pre-stated expectation

`inherited-f4` and `f4-f2` decide each target below `m4gb-f2` on every cell: they stop at a degree-bounded decision while M4GB computes the full reduced basis of the same ideal, and M4GB pays a process start, a file round trip and its own table initialisation per call (tens of milliseconds). The ratio M4GB / F4 should grow with the number of unknowns. Falsifier: `m4gb-f2` median wall below `f4-f2` on any cell with at least 24 unknowns.

Results

`RESULTS.md` (table), `results.svg` / `results.png` (graph), `results.json`, `REPORT.pdf`; raw per-cell JSON under `results/` (the `.log` files are the same JSON as captured on stdout). Regenerate with `./make_report.sh`. 25 measured cells (19 at `m = 2`: Koblitz instances exist at `n = 11, 13, 15, 17, 19, 23, 29, 31` only; the random family at every degree; 6 at `m = 3`), 6 targets each, 2 repetitions, every decided answer of every engine equal to the reference's: **0 disagreements**. All numbers `[pcc:unverified]`, one thread, local container.

- **Answer to the question.** Per decomposition call, `m4gb-f2` is faster `[pcc:unverified]` than the in-house full-basis `f4-f2` from about 22 unknowns upward and never faster than the degree-bounded `inherited-f4` or the exhaustive reference. At 24 unknowns (Koblitz `n = 23`) M4GB takes 489 ms against 2059 ms for `f4-f2` (0.24×); at 28 unknowns (random `n = 27`) 3.3 s against 40.7 s (0.08×); at 30 and 32 unknowns `f4-f2` exhausts the 60 s budget on 3 to 6 of 6 targets while M4GB decides every target (7.8 to 28 s). Below 20 unknowns M4GB is slower (9 to 40 ms against 1 to 30 ms): a process start, a file round trip and the solver's table set-up are paid on every call.
- **The pre-stated falsifier fired.** The expectation was that `f4-f2` stays below `m4gb-f2` on every cell; it does not from 22 unknowns on. The other half held: `inherited-f4` is 10 to 90× faster than M4GB on every cell (242 ms against 21.8 s at 32 unknowns), and `fes-f2` is 7 to 200× faster. The decomposition oracle stays `inherited-f4`.
- **Scaling.** M4GB's ratio to the exhaustive reference falls from about 160× at 12 unknowns to 7 to 10× at 32: its cost grows about 2.5 to 3× per two unknowns against 4× for the Gray-code search. On that trend the crossover with `fes-f2` lies beyond the descent's 64-variable cap unless the trend bends; `inherited-f4` grows slower still (about 2× per two unknowns here).
- **Degrees.** `f4-f2` reaches degree 4 to 5 for its full basis, M4GB's highest critical-pair degree is 3 to 4, `inherited-f4` stops at 3.
- **`m = 3` (cubic systems, 12 to 15 unknowns).** M4GB is 45 to 87× slower than `f4-f2` at 12 unknowns and over budget on 4 to 6 of 6 targets at 15; this is the regime where the earlier profile put the time in M4GB's Gebauer–Möller pair update, not in the dense kernels this PR patched.
- **Lost first attempt.** The first run (8 targets, 3 repeats, 300 s budget, one `ic` invocation for all cells) was killed by the session's two-hour cap before `ic descent` wrote its single output file; nothing of it was recorded. The per-cell launcher replaced it.

Open threads

- Isolab re-run of `run_ladder.sh` on a pinned image so the comparison can carry a pcc claim; this container shared CPU with a Rust build at times.
- Extend the `m = 2` ladder past 32 unknowns (the descent holds 64) with a larger budget, to see whether M4GB's slope against `fes-f2` holds.
- M4GB's per-call overhead (about 8 ms) would matter for an oracle that is called millions of times; a persistent-process mode would remove it.

Median over targets of the per-target median wall, ms, one call. Ratio columns: engine / `f4-f2` and engine / reference on the same cell. `dec` = targets decided in every repetition / targets; `ob` = targets with a budget verdict; `agree` = every decided answer equals the reference.

family	n	n'	m	vars	ref	`fes-f2` ms (dec, ob, agree, Dmax)	`f4-f2` ms (dec, ob, agree, Dmax)	`inherited-f4` ms (dec, ob, agree, Dmax)	`m4gb-f2` ms (dec, ob, agree, Dmax)	m / f
K	11	6	2	12	`fes-f2`	0.06 (6/6, 0, y, -)	1.22 (6/6, 0, y, 4)	0.60 (6/6, 0, y, 3)	8.76 (6/6, 0, y, 3)	7
R	11	6	2	12	`fes-f2`	0.07 (6/6, 0, y, -)	1.55 (6/6, 0, y, 4)	0.72 (6/6, 0, y, 3)	11.6 (6/6, 0, y, 3)	7
K	13	7	2	14	`fes-f2`	0.09 (6/6, 0, y, -)	2.78 (6/6, 0, y, 4)	1.02 (6/6, 0, y, 3)	13.1 (6/6, 0, y, 3)	4
R	13	7	2	14	`fes-f2`	0.10 (6/6, 0, y, -)	3.81 (6/6, 0, y, 4)	1.12 (6/6, 0, y, 3)	20.6 (6/6, 0, y, 3)	5
K	15	8	2	16	`fes-f2`	0.22 (6/6, 0, y, -)	9.39 (6/6, 0, y, 4)	2.58 (6/6, 0, y, 3)	24.3 (6/6, 0, y, 4)	2
R	15	8	2	16	`fes-f2`	0.17 (6/6, 0, y, -)	7.27 (6/6, 0, y, 4)	1.84 (6/6, 0, y, 3)	24.3 (6/6, 0, y, 3)	3
K	17	9	2	18	`fes-f2`	0.32 (6/6, 0, y, -)	32.5 (6/6, 0, y, 4)	2.94 (6/6, 0, y, 3)	41.6 (6/6, 0, y, 3)	1
R	17	9	2	18	`fes-f2`	0.39 (6/6, 0, y, -)	26.7 (6/6, 0, y, 4)	2.96 (6/6, 0, y, 3)	39.2 (6/6, 0, y, 3)	1
K	19	10	2	20	`fes-f2`	0.90 (6/6, 0, y, -)	31.7 (6/6, 0, y, 4)	4.56 (6/6, 0, y, 3)	39.5 (6/6, 0, y, 3)	1
R	19	10	2	20	`fes-f2`	0.92 (6/6, 0, y, -)	66.7 (6/6, 0, y, 4)	6.12 (6/6, 0, y, 3)	55.9 (6/6, 0, y, 3)	0

family	n	n’	m	vars	ref	fes-f2 ms (dec, ob, agree, Dmax)	f4-f2 ms (dec, ob, agree, Dmax)	inherited-f4 ms (dec, ob, agree, Dmax)	m4gb-f2 ms (dec, ob, agree, Dmax)	m / f
R	21	11	2	22	fes-f2	3.94 (6/6, 0, y, -)	251.7 (6/6, 0, y, 5)	9.54 (6/6, 0, y, 3)	270.1 (6/6, 0, y, 4)	1
K	23	12	2	24	fes-f2	11.4 (6/6, 0, y, -)	2058.9 (6/6, 0, y, 5)	18.2 (6/6, 0, y, 3)	489.3 (6/6, 0, y, 4)	0
R	23	12	2	24	fes-f2	11.7 (6/6, 0, y, -)	1461.8 (6/6, 0, y, 5)	16.6 (6/6, 0, y, 3)	367.8 (6/6, 0, y, 4)	0
R	25	13	2	26	fes-f2	63.0 (6/6, 0, y, -)	11500.9 (6/6, 0, y, 5)	32.1 (6/6, 0, y, 3)	1144.0 (6/6, 0, y, 4)	0
R	27	14	2	28	fes-f2	180.7 (6/6, 0, y, -)	40709.4 (6/6, 0, y, 5)	51.5 (6/6, 0, y, 3)	3293.6 (6/6, 0, y, 4)	0
K	29	15	2	30	fes-f2	730.8 (6/6, 0, y, -)	14260.1 (3/6, 3, y, 4)	107.2 (6/6, 0, y, 3)	7761.1 (6/6, 0, y, 4)	0
R	29	15	2	30	fes-f2	738.6 (6/6, 0, y, -)	14660.3 (1/6, 5, y, 4)	114.5 (6/6, 0, y, 3)	9860.8 (6/6, 0, y, 4)	0
K	31	16	2	32	fes-f2	2962.8 (6/6, 0, y, -)	38374.8 (3/6, 3, y, 4)	242.3 (6/6, 0, y, 3)	21758.6 (6/6, 0, y, 4)	0
R	31	16	2	32	fes-f2	2923.0 (6/6, 0, y, -)	n/a (0/6, 6, y, -)	251.3 (6/6, 0, y, 3)	28211.3 (6/6, 0, y, 4)	
K	11	4	3	12	f4-f2	declined	63.5 (6/6, 0, y, 8)	28.0 (6/6, 0, y, 6)	5536.2 (6/6, 0, y, 7)	87
R	11	4	3	12	f4-f2	declined	141.5 (6/6, 0, y, 8)	29.6 (6/6, 0, y, 6)	6380.5 (6/6, 0, y, 7)	45
K	13	5	3	15	f4-f2	declined	13462.4 (6/6, 0, y, 8)	220.2 (6/6, 0, y, 6)	45718.2 (2/6, 4, y, 7)	3
R	13	5	3	15	f4-f2	declined	9296.1 (6/6, 0, y, 8)	216.6 (6/6, 0, y, 6)	44620.3 (2/6, 4, y, 7)	4
K	15	5	3	15	f4-f2	declined	6420.6 (6/6, 0, y, 8)	225.3 (6/6, 0, y, 6)	n/a (0/6, 6, y, -)	

family	n	n’	m	vars	ref	fes-f2 ms (dec, ob, agree, Dmax)	f4-f2 ms (dec, ob, agree, Dmax)	inherited-f4 ms (dec, ob, agree, Dmax)	m4gb-f2 ms (dec, ob, agree, Dmax)	m / f
R	15	5	3	15	f4-f2	declined	6345.7 (6/6, 0, y, 8)	205.3 (6/6, 0, y, 6)	n/a (0/6, 6, y, -)	